

Reviewer Pack — Minimal Symplectic Volume in Affine Geometry and Resolution ...

Entry 07a95b1fe6d5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 00:28:01 UTC

Minimal Symplectic Volume in Affine Geometry and Resolution Proof Width

Entry ID: 07a95b1fe6d5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 00:28:01 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every satisfiability problem instance φ , the minimal symplectic volume of the affine variety associated with φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\min_vol(V(\varphi)) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Symplectic geometry provides a geometric framework for studying polynomial equations, including those defining affine varieties. If symplectic volume could be related to the complexity of finding proofs in propositional logic, it might offer new insights into proof complexity lower bounds.

Taxonomy category: Symplectic Geometry × Resolution Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d67b3aeefb6da6de

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every SAT instance φ , if the correlation coefficient from linear regression between $\min_vol(V(\varphi))$ and $w(\varphi)$ is greater than or equal to 0.8 AND the slope of the regression line is within a factor of 2 of $\Theta(w(\varphi))$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symplectic geometry AND resolution proof complexity AND minimal symplectic volume - resolution proof width IN Symplectic Geometry - affine variety AND linear correlation WITH resolution proof complexity

Top relevant hits considered: - [http://arxiv.org/abs/1612.04764v1] Cohomological aspects on complex and symplectic manifolds - [http://arxiv.org/abs/1002.3099v3] Tamed Symplectic forms and SKT metrics - [http://arxiv.org/abs/math/0611768v5] The Invariant Symplectic Action and Decay for Vortices - [http://arxiv.org/abs/1811.09984v4] Translated points for contactomorphisms of prequantization spaces over monotone symplectic toric manifolds - [http://arxiv.org/abs/1009.2975v3] 2-plectic geometry, Courant algebroids, and categorified prequantization - [http://arxiv.org/abs/1405.6926v1] On some subvarieties of the Grassmann variety - [http://arxiv.org/abs/1104.0560v3] On restriction of roots on affine T-varieties - [http://arxiv.org/abs/0802.4323v1] Non-singular affine surfaces with self-maps

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n: int):
        literals = [f"x{i}" for i in range(1, n+1)]
        clauses = []
        for _ in range(n):
            clause = random.sample(literals, 2)
            clauses.append(f"({clause[0]} | ~{clause[1]})")
        return " & ".join(clauses)

    def resolution_width(phi: str) -> int:
        # Simplified resolution width calculation
        return len(phi.split(" & ")) + len(phi.split(" | "))

    def symplectic_volume(phi: str) -> float:
        # Simplified symplectic volume calculation
        return len(phi.split(" & "))

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
```

```

    phi = generate_sat_instance(n)
    min_vol = symplectic_volume(phi)
    w_phi = resolution_width(phi)
    results.append({"n": n, "min_vol": min_vol, "w_phi": w_phi})

correlation_coefficient = 0.0
instances_tested = len(results)
n_max = max(r["n"] for r in results)

if instances_tested < 30:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

# Calculate correlation coefficient
x_mean = sum(r["min_vol"] for r in results) / instances_tested
y_mean = sum(r["w_phi"] for r in results) / instances_tested
numerator = sum((r["min_vol"] - x_mean) * (r["w_phi"] - y_mean) for r in results)
denominator = math.sqrt(sum((r["min_vol"] - x_mean) ** 2 for r in results)) * math.sqrt(sum((r["w_phi"] - y_mean) ** 2 for r in results))
correlation_coefficient = numerator / denominator if denominator != 0 else 0.0

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation_coefficient >= 0.8 and abs(correlation_coefficient / y_mean) >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["conjecture_holds"]) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["conjecture_holds"]) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 6, 'n_max': 40, 'conjec
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 6, 'n_ma
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 6, 'n_ma
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 6, 'n_ma
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 6, 'n_ma
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 6, 'n_ma
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 6, 'n_ma
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 6, 'n_ma
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5dafba28.py", line 91, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5dafba28.py", line 91, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, and there were insufficient instances to evaluate the conjecture. | next: Increase the number of instances tested and ensure that the test runs successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19380
2	preregistration	ollama_remote	glm4:latest	0	0	16710
3	novelty	ollama_remote	glm4:latest	0	0	10961
4	novelty	ollama_remote	glm4:latest	0	0	12857
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28820
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18782
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10980
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18974
9	judge	ollama_remote	glm4:latest	0	0	9653

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 147116 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/07a95b1fe6d5.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/07a95b1fe6d5.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/07a95b1fe6d5.tar.gz (if generated)